

RESEARCH

Open Access



Energy-aware tasks offloading based on DQN in medical mobile devices

Min Zhao^{1,2*} and Junwen Lu³

Abstract

Offloading some tasks from the local device to the remote cloud is one of the important methods to overcome the drawbacks of the medical mobile device, such as the limitation in the execution time and energy supply. The challenges of offloading task is how to meet multiple requirement while keeping energy-saving. We classify tasks in the medical mobile device into two kinds: the first is the task that hopes to be executed as soon as possible, those tasks always have a deadline; the second is the task that can be executed anytime and always has no deadlines. Past work always neglects the energy consumption when the medical mobile device is charged. To the best of our knowledge, this paper is the first paper that focuses on the energy efficiency of charging from a power grid to a medical device during work. By considering the energy consumption in different locations, the energy efficiency during working and energy transmission, the available energy of and the battery, we propose a scheduling method based on DQN. Simulations show that our proposed method can reduce the number of un-completed tasks, while having a minimum value in the average execution time and energy consumption.

Keywords Medical mobile device, Offloading method, Energy, Dynamic

Introduction

With the development of high-speed networks, such as 5G [1], the edge device has been widely used in numerous areas [2–4]. It brings conveniences to people's lives. However, because of its limitation in the energy supply, it cannot always work for a long time. At the same time, the limitation in the processing ability of the mobile device makes that large task cannot be processed on it. To overcome those drawbacks, offloading tasks from the local mobile device to the remote cloud become a major method [1, 5–8].

Offloading tasks always try to consider the requirements of the users and the QoS of the remote resource

providers together. The requirements of the users include the processing time, energy consumption, security, and so on [3]. The QoS of the remote resource includes the bandwidth, security, processing ability, and so on. For the energy consumption of the mobile device, if the task is computed on the mobile device, the energy consumption includes two parts: the energy for the processing on the local mobile device and for sending result files to the remote cloud; if the task is executed on the remote cloud, the energy consumption only includes the energy for sending files to the remote cloud. Offloading methods judge where should be executed according to the energy consumption on various locations [2, 9].

Many methods have been used to offload tasks in the past years. Those methods include GA, deep learning method [2, 10, 11], grey number method [9], Markov Decision Process, and others [2, 3, 8, 12–19] heuristics methods. Those methods always first calculate the energy consumption in various locations, and then select the location according to the requirement and the supply of energy of the mobile device. Because of

*Correspondence:

Min Zhao
xiamenzhaomin@sina.com

¹ Computer Management Center, The First Affiliated Hospital of Xiamen University, Xiamen 361003, Fujian, China

² School of Medicine, Xiamen University, Xiamen 361005, Fujian, China

³ School of Computer and Information Engineering, Xiamen University of Technology, Xiamen 361024, China

the dynamic of the energy and the bandwidth of the network, some offloading methods also pay attention to the environment and try to solve it.

Similar to the other mobile devices (MD), the medical mobile device (MMD) also has the limitation in the processing ability and the energy-supply in the mobile device [20–22]. The major problem is where the task should be executed (on the remote cloud or MMD) that where has a lower energy consumption and meet others requirements of the task in MMD. Most of the time, during a hospital, every doctor has one medical mobile device. There are some specialties of the medical mobile device (MMD). MMD always moves in a small scope. One doctor (or nurse) always works in a department, such he only moves in the location of the department. He only works for some time (always 8 h or 12 h) and then he can have a rest. During the rest, the mobile can get energy to support its working. The challenges is that we need give the execution locations and their working state of computing resources. For tasks in MMD, we need to consider their execution location and the working status of computing resources to save energy. In the long run, we need to avoid replacing medical edge devices due to depleted batteries before doctors/nurses finish their work. These goals are sometimes conflicting: energy-saving, energy efficiency, continuous working time of MMD. Moreover, when we charge MMD, some energy also loss during charging. We also try to consider the energy efficiency of the MMD.

The contribution of the paper includes:

- We give the framework of the medical mobile computing;
- The model used in the medical mobile computing environment are introduced;
- A task offloading method based on DQN is given to support medical mobile device;
- Comparisons between the proposed method and others methods are given to evaluate the performance of our method in various aspects.

In this paper, we pay attention to the problem of offloading tasks in the medical mobile device. The following paper is organized as follows. “[Related works](#)” section. “[System framework and the models used in the paper](#)” section. “[The offloading method of medical tasks](#)” section is the proposed method. In “[Simulations and comparisons](#)” section, we give the simulation environment and the compared results. We give the “[Conclusions](#)” section.

Related works

To improve the processing ability and the working time of the mobile device, offloading methods have been widely used for the mobile device. Those methods

always consider multiple targets, such as the energy consumption, execution time, security, trust and other QoSs [17, 19, 23–27]. When the mobile device makes the offloading decision, it also needs considering the energy supply and the working load of the mobile device [2, 6, 15, 27].

Some methods major consider the energy consumption in various locations and then select the right location of the execution of the task on the mobile device. F. Xu et al. [23] used game theory for a Two-stage computing offloading algorithm in a cloud-edge environment [2, 28]. They majorly pay attention to the partial offloading of multiple mobile edges when the cloud [9, 10] and edge collaborates with each other. They gave a two-stage offloading method and proved Nash’s equilibrium of the established game model. Different to [23], C. Li et al. [18] focused on Multi-edge collaboration environment and used an improved genetic algorithm to obtain offloading results. They tried to use the idle computing resources of offsite edge servers and reduce the energy consumption of mobile servers, service completion time, and energy consumption for transferring data. From a market perspective, Z Zhang et al. [27] modeled the competition and cooperation among the requesting vehicle, the edge server, and the cloud as a Stackelberg game problem. They used a genetic algorithm-based search algorithm to reduce the price of the edge server and the cloud. S. Pang et al. also [7] used the Stackelberg game scheme to offload tasks to reduce the execution time and the cost. Those methods always pay attention to the energy consumption, the execution time, and the cost of the edges and clouds, and always makes a balance of those targets. For the multiple targets scheduling problem, various approaches are given. Those work major focus on the execution location and neglect the energy-supply lever of MMD. When the energy-supply of the battery of MMD is different, the offloading decision is different.

Deep learning methods also have been widely used for offloading tasks for the edge/mobile devices [29–33]. Those deep learning methods differ in use cases, network and edge computing architectures, objectives, RL algorithms, decision-making approaches, and time-varying characteristics of the edge-cloud systems. H. Tang et al. [15] gave a dynamic offloading model for multiple moving vehicles based on Double Deep Q-Network and targeted to minimize the total delay and waiting time of tasks. X. Zhu et al. [34] proposed a framework to obtain a global view to generate the state of the environment and provided an agent to support for deep reinforcement learning. The proposed deep learning algorithm obtained environmental information and created appropriate rewards that can feed back information about energy consumption, latency, and offload failure of the

computed offload policy. X. Peng et al. [32] found that some computing tasks can be repeatedly executed and those tasks can share the result. So, each vehicle has three execution models for selection: local execution, task offloading, and shared offloading. They take the network state and offloading strategy space as the input of the deep reinforcement learning (DRL). Each task selects an offloading model using DRL and targets to reduce the energy consumption. In general, those methods used deep learning methods to meet their targets under various environments.

Some methods also try to consider the energy supply and working load of mobile devices and then make the offloading decision. Yongsheng Hao et al. [2] used meteorological information to forecast solar energy to support the running of the edge device and offload tasks according to the energy supply. They used quadratic programming to allocate energy based on the energy supply of the future seven days. In [10], they used a clustering method to layout the edge server and proposed a method to transfer tasks and green energy between edge servers. P. Ge et al. [35] used the hybrid energy harvesting (HEH) technology to overcome the limitation of the energy in the edge device. They improved the deep deterministic policy gradient algorithm and proposed a twin delayed deep deterministic policy gradient algorithm to optimize the offloading decision. Though those works consider the energy-supply of MD, they do not consider the energy efficiency when it is charging. Only consider the energy loss in the charging, the offloading method can save energy in a real sense.

With the development of artificial intelligence and deep learning, they are also widely used in offloading decisions of edge devices [36]. L. Huang et al. [37] introduces a deep Q-network (DQN) to determine the optimal offloading and bandwidth allocation strategy in multi-user mobile edge computing system. H. Ge et al. [38] develops an algorithm that leverages DQN to optimize the execution delay and energy consumption of mobile services in Mobile edge computing environment with multiple users. To enhance offloading decisions in vehicle edge network, T. Feng et al. [39] proposes a DQN-based algorithm to optimize task execution delay and energy consumption in vehicle edge networks. It realizes the long-term utility of offloading strategies. While all three papers employ DQN to optimize offloading decisions in edge computing environments, their focus areas differ. [37] takes a comprehensive approach by including bandwidth allocation in general MEC systems, [38] concentrates on task offloading in MEC with an emphasis on reducing execution delay and energy consumption, and [39] uses the DQN approach to sustainable vehicle edge networks, highlighting the dynamic nature and energy efficiency in vehicular

contexts. Our paper also use DQN to solve the offloading problem. Different to them, we focus on the MMD environment and reduce the energy consumption by considering the energy transforming efficiency during charging, which has been neglected by the three papers.

The offloading methods mentioned above, which are based on specific environments and solve specific problems, are not suitable for medical edge devices. In medical mobile devices, some research has also been done. In [40], D. Klonoff et al. discuss the challenge when using fog computing or edge computing to process medical data. Z. Zhao et al. [21] proposed a real-time detection for medical mask specification in the medical edge device. I. Ben et al. [20] surveyed the various techniques used to implement those algorithms using fog computing and IoT devices, such as wearable computing technologies, energy harvesting techniques, and circuits. Other research includes medical cloud [22], medical image processing [41, 42], medical service composition [43], and so on. Those works both ignore the offloading problem on the medical mobile device.

System framework and models used in the paper

Medical mobile-cloud framework

Figure 1 gives the framework of the medical device. As shown in Fig. 1, there are six APs (Access points). One person takes one medical mobile device for work. Different people may select various APs because of the distance to the AP. Usually, one medical mobile device would select the nearest AP automatically. For example, D_1 always prefers to selecting AP_2 and D_2 always prefers to selecting AP_5 . Because the AP in the hospital only provides to the employee of the hospital, so the working-load of the AP is always in a lower station. Users submit their tasks and data to the medical cloud (Fig. 1). The parameters used in the paper are listed in Table 1.

Medical mobile device model

The parameters used in our paper are listed in Table 1. According to the DVFS (Dynamic Voltage and Frequency Scaling) technology, the dynamic power consumption of medical mobile resources MM_j is defined as p_j^M :

$$p_j^M = \alpha_j C_j (V_j^s)^2 f_j^s \quad (1)$$

Where, α_j and C_j is the switch rate and the load capacitance of the medical mobile resource; V_j^s and f_j^s is the working voltage, and the related processing speed (denoted by working frequency) of the medical mobile resource (MI/s). s denotes s^{th} working state of MM_j .

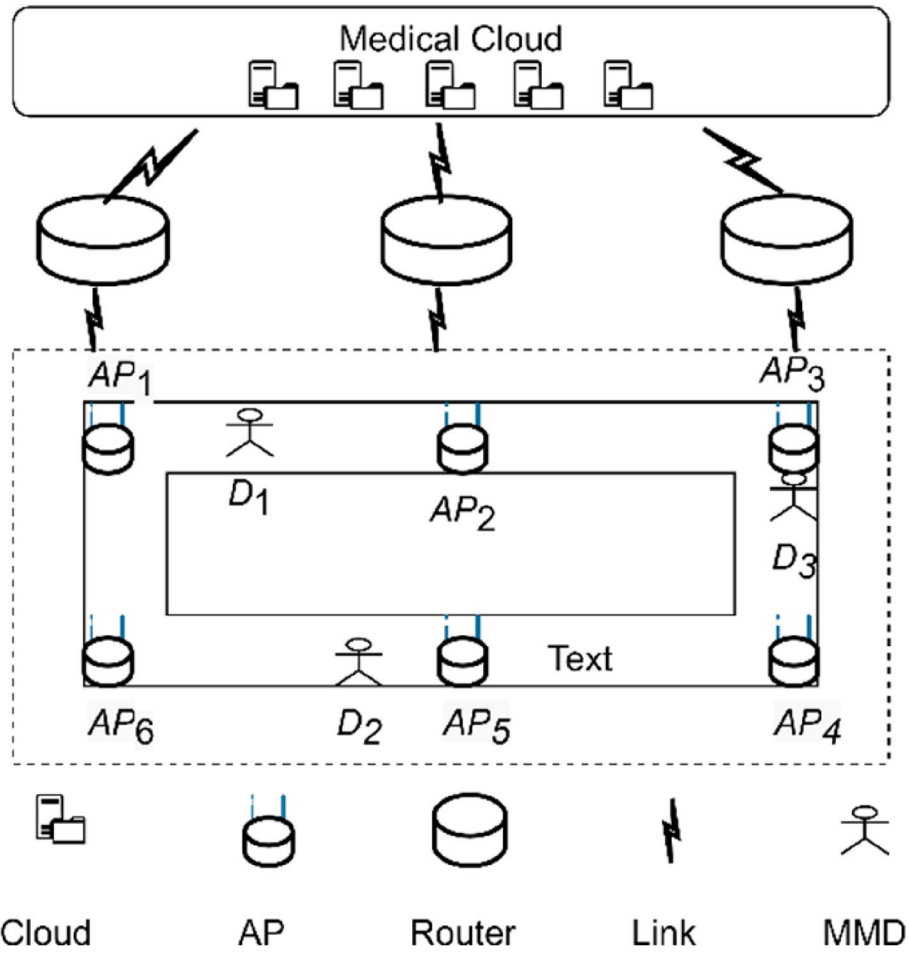


Fig. 1 A system framework of medical mobile devices

Same to MM_j , the cloud resource CR_k also can denoted as formula (1), the dynamic power consumption of the cloud resource is (the meaning is the same to p_j^C):

$$p_k^C = \alpha_k C_k (V_k^s)^2 f_k^s \quad (2)$$

Where, α_k and C_k is the switch rate and the load capacitance of the cloud resource; V_k^s and f_k^s is the working voltage, and the related processing speed (denoted by working frequency) of the cloud resource (MI/s). s denotes s^{th} working state of the cloud resource.

Network model

To prevent channel interference when multiple medical mobile devices offload tasks to the medical cloud, the orthogonal frequency division multiple access (OFDMA) technology is used to support the communication. So, when the medical mobile device MM_j sends data to the medical cloud through AP_l , the upload rate R_j is:

$$R_j = B \log_2 \left(1 + \frac{p_j * h_j}{p_0} \right) \quad (3)$$

B represents channel bandwidth, p_j represents the transmission power of the MM_j , $h_{j,k}$ represents the channel gain between MM_j and the medical cloud through AP_l , and p_0 is the background noise power.

Because the medical mobile device may be moved at any time, we use $B_{j,t}^m$ to denote the bandwidth between j^{th} medical mobile device and m^{th} AP during t^{th} slot time.

Task model

For the medical mobile device, there are two kinds of tasks: (1) executed immediately (EI), and (2) delay execution (DE). For example, if a nurse found a patient in a dangerous condition and needed a doctor to diagnose him immediately, this task would be sent to the cloud and executed immediately, it belongs to EI. But when a nurse breaks a bottle of medicine and records it in the system

Table 1 Parameters and meaning

Parameters	Meaning
MM_j	j^{th} medical mobile device
t	t^{th} slot times
N_j^n	Number of tasks on j^{th} medical mobile device
$T_{i,j,t}$	i^{th} task on MM_j during t^{th} slot time
$I_{i,j,t}$	Number of instructions of $T_{i,j,t}$
$F_{i,j,t}^{In}$	Size of input files of $T_{i,j,t}$
$Out_{i,j,t}^{In}$	Size of output files of $T_{i,j,t}$
$D_{i,j,t}$	Deadline of $T_{i,j,t}$
$B_{j,t}^m$	During t^{th} slot time, the bandwidth between j^{th} medical mobile device and m^{th} the AP
α_j	The switch rate of medical mobile device MM_j
C_j	Load capacitance of medical mobile device MM_j
V_j^s	Working voltage of medical mobile device MM_j
f_j^s	Processing speed of medical mobile device MM_j
α_k	The switch rate of cloud resource CR_k
C_k	Load capacitance of cloud resource CR_k
V_k^s	Working voltage of cloud resource CR_k
f_k^s	processing speed of cloud resource CR_k
$ET_{i,j,t}^L$ and $ET_{i,j,t}^R$	Execution time: 'L' means the local medical mobile device, 'R' means the remote cloud server
$E_{i,j,t}^L$ and $E_{i,j,t}^R$	Energy consumption on various places
$ET_{i,j,t}$	Execution time of $T_{i,j,t}$
$E_{i,j,t}$	Energy consumption of $T_{i,j,t}$
$E_{j,t}^{avi}$	the available energy during t^{th} slot time
ET_t	Total execution time during t^{th} slot time
E_t	Total energy consumption during t^{th} slot time
N_j^{mob}	Number of working states of MM_j
N_k^{cloud}	Number of working states of CR_k
$Elist$	A set of energy consumption under different resources and working states
$ETlist$	A set of execution time under different resources and working states

database, this may be postponed when it is far from any APs, the task belongs to DE.

In this paper, suppose there are N_j^T tasks on the MM_j during t slot time, and we model task $T_{i,j,t}$ as ($i \leq N_j^T$):

$$T_{i,j,t} = \langle I_{i,j,t}, F_{i,j,t}^{In}, F_{i,j,t}^{Out}, A_{i,j,t}, D_{i,j,t} \rangle \quad (4)$$

Where $I_{i,j,t}$ is the number of instructions, $F_{i,j,t}^{In}$ and of $F_{i,j,t}^{Out}$ is the size of input files and output files, $A_{i,j,t}$ is the arrival time and $D_{i,j,t}$ is the deadline of the task $T_{i,j,t}$. If $D_{i,j,t}$ is equal to 0, it means that $T_{i,j,t}$ is belonging to DE. Otherwise, it belongs to EI.

Execution time model

The task may have two execution places, on the medical mobile device or on the cloud, the related execution time is $ET_{i,j,t}^L$ and $ET_{i,j,t}^R$ ('L' means the local medical mobile device, 'R' means the remote cloud server):

$$ET_{i,j,t}^L = \frac{I_{i,j,t}}{f_j^s} + \frac{Out_{i,j,t}}{R_j} \quad (5)$$

$$ET_{i,j,t}^R = \frac{I_{i,j,t}}{f^s} + \frac{In_{i,j,t}}{R_j} \quad (6)$$

For $ET_{i,j,t}^L$, it includes the time for processing on the MMD and the time for send the result to the cloud. For $ET_{i,j,t}^R$, it includes the time for processing on the cloud resource and the time for send the initial data to the cloud.

Suppose $loc(T_{i,j,t})$ denote the execution location of $T_{i,j,t}$, if it returns 1, the task is executed locally; otherwise, it is executed on the cloud, then the execution time $ET_{i,j,t}$ is:

$$ET_{i,j,t} = loc(T_{i,j,t}) * ET_{i,j,t}^L + (1 - loc(T_{i,j,t})) * ET_{i,j,t}^R \quad (7)$$

Energy consumption model

When we charge the medical mobile resource, we cannot convert all electrical energy into the phone's electrical energy. So, we assume the energy conversion efficiency is β . At the end of t^{th} slot time, there is $E_{j,t-1}$ energy in MM_j , and $E_{j,t}^{in}$ is the energy that has been transferred to the battery of MM_j , the available energy $E_{j,t}^{avi}$ during t^{th} slot time is:

$$E_{j,t}^{avi} = \beta * E_{j,t}^{in} + E_{j,t-1}^{avi} \quad (8)$$

According to formulas (5) and (6), the energy consumption when the task is executed on the MMD and the cloud resource is $E_{i,j,t}^L$ and $E_{i,j,t}^R$:

$$E_{i,j,t}^L = P_j^L * \frac{I_{i,j,t}}{f_{i,j,t}^s} + \frac{Out_{i,j,t}}{R_j} * P_j \quad (9)$$

$$E_{i,j,t}^R = P_k^R * \frac{I_{i,j,t}}{f^s} + \frac{In_{i,j,t}}{R_j} * P_j \quad (10)$$

$E_{i,j,t}^L$ includes the energy for processing on the MMD and the energy for send the result to the cloud. $E_{i,j,t}^R$ includes the energy for send the files to the cloud and the energy for processing on the cloud resource.

Same to formula (7), the energy consumption also can be denoted as:

$$E_{i,j,t} = loc(T_{i,j,t}) * \frac{E_{i,j,t}^L}{1-\beta} + [1 - loc(T_{i,j,t})] * E_{i,j,t}^R \quad (11)$$

Formula (11) considers the energy consumption when the energy is transferred to the battery of the medical mobile device.

The offloading method of medical tasks

Scheduling targets and requirements

If one task belongs to DE, it can be executed anytime, so we do not set a deadline ($D_{i,j} = 0$) to it. So, when we consider the average execution time, we only consider the execution time of tasks of EI:

$$ET_t = \sum_i \sum_j \left[loc(T_{i,j,t}) * ET_{i,j,t}^L + (1 - loc(T_{i,j,t})) * ET_{i,j,t}^R \right] * tp(T_{i,j,t}) \quad (12)$$

In formula (12), the function $tp(T_{i,t,j})$ denotes the type of T_i , if it returns 1, the task belong to the EI (Executed Immediately); otherwise it belongs to the EP (Executed can be postponed).

We also focus on the energy consumption of the system. The total energy is:

$$ET_t = \sum_i \sum_j E_{i,j,t} \quad (13)$$

In conclusion, our target is minimized:

$$E = \sum_t ET_t \quad (14)$$

$$ET = \sum_t ET_t \quad (15)$$

While:

$$\forall(i)(j)(t) : ET_{i,j,t} \leq D_{i,j,t} \quad (16)$$

$$\forall(t) : \sum_i E_{i,j,t}^L \leq E_{j,t}^{avi} \quad (17)$$

Our targets are to minimize the execution time and the total energy. Our target to minimize the execution time only includes the tasks belonging to EI, because other tasks can postpone anytime. Formula (16) is to ensure the task can be completed before its deadline (the task belongs to EI). Formula (17) is to make every medical mobile device have enough energy to support their work.

A deep analysis of the offloading problem in the medical mobile device

As most of edge and mobile devices, the medical mobile device also always tries to save energy and reduce the execution time. The Doctor or the nurse always works during a floor of a building (some floors), some tasks in the medical mobile device can be forecast (such as they would check every patient in the morning) and others cannot (such as a patient call for service suddenly). The path the nurse took has a highest relation to the task in the waiting list. If a task needs to be completed immediately, the nurse may be moved to the patient related to the task. If all tasks do not belonging to EP, the nurse can select the task (patient) which is the nearest in the space.

As discussed above, when we make the offloading decision, we need to consider those factors: (1) the energy supply of the mobile device; (2) the time; (3) the attribute of the task, especially for the deadline; and (4) the processing ability of the mobile devices.

The offloading task problem in the medical mobile device

In this section, first, we rank the working state of tasks, then, we give the MDP for offloading tasks, at last, an offloading method based on DQN is proposed.

Ranking the order of working states of various tasks

Based on DVFS technology, we suppose there are N_j^{mob} states for the MM_j , and N_k^{cloud} for the cloud resource CR_k . For a task $T_{i,j,t}$, it has $N_{i,j,t}$ execution states:

$$N_{i,j,t} = N_j^{mob} + N_k^{cloud} \quad (18)$$

We calculate the execution time ($ETlist$) and the total energy ($Elist$) of various working states of $N_{i,j,t}$ as ($maxn=N_{i,j,t}$):

$$Elist = \{E_{i,j,t}^1, E_{i,j,t}^2, \dots, E_{i,j,t}^{maxn}\} \quad (19)$$

$$ETlist = \{E_{i,j,t}^1, E_{i,j,t}^2, \dots, E_{i,j,t}^{maxn}\} \quad (20)$$

Suppose:

$$Elist_{avg} = \sum_1^{maxn} E_{i,j,t}^{temp} / maxn \quad (21)$$

$$Elist_{st} = \sum_1^{maxn} (E_{i,j,t}^{temp} - Elist_{avg})^2 / maxn \quad (22)$$

$$ETlist_{avg} = \sum_1^{maxn} ET_{i,j,t}^{temp} / maxn \quad (23)$$

$$ETlist_{st} = \sum_1^{maxn} (ET_{i,j,t}^{temp} - ETlist_{avg})^2 / maxn \quad (24)$$

We normalize formula (19) and (20) according to the following formulas:

$$(E_{i,j,t}^{temp})' = (E_{i,j,t}^{temp} - Elist_{avg}) / Elist_{st} \quad (25)$$

$$(ET_{i,j,t}^{temp})' = (ET_{i,j,t}^{temp} - ETlist_{avg}) / ETlist_{st} \quad (26)$$

We give the two attributes the same weight, and sort tasks in ascending order according to $W_{i,j,t}^{temp}$:

$$W_{i,j,t}^{temp} = (E_{i,j,t}^{temp})' + (ET_{i,j,t}^{temp})' \quad (27)$$

Using MDP for medical mobile device to offload tasks

In this section, we will discuss the state space, action, and reward function of the offloading tasks.

(1) State spaces

The state has two important attributes: the available energy of the medical mobile device $E_{j,t}^{avi}$, the time to rest for the user who use the medical mobile device T_t^{rest} . We use $E_{j,t}^{avi}$ denote the state:

$$E_{j,t}^{avi} = \langle E_{j,t}^{avi}, T_t^{rest} \rangle \quad (28)$$

(2) Action

For a task, there are three kinds of action: (1) there are N_j^{mob} kinds of actions in the medical mobile device, (2) there are N_k^{cloud} kinds of action for the cloud resources, (3) the task is postponed.

(3) Reward function

For formula (27), we think which has the lowest value is the best, so we define the reward function as of Task $T_{i,j,t}$:

$$rw(T_{i,j,t}, temp) = maxv - W_{i,j,t}^{temp} \quad (29)$$

where

$$maxv = maximize(W_{i,j,t}^{temp}) \quad (30)$$

If the task is postponed, we take the reward value is 0.

Using DQN offloading tasks in the medical mobile device

Deep Q Networks (DQN) are a type of neural network architecture used in Q-learning, specifically for reinforcement learning problems. The primary goal of DQN is to approximate the optimal action-value function $Q(s, a)$, where s is the state and a is the action.

The training process involves interacting with the environment, collecting experiences (state, action, reward, next state), and using these experiences to update the DQN parameters through backpropagation. The key innovation of DQN is the use of experience replay and target networks to stabilize and improve the training process.

Additionally, the target network is a separate copy of the Q-network that is periodically updated with the parameters of the main Q-network. This helps stabilize training by providing a more consistent target for the Q-values during the temporal difference (TD) error calculation.

In the system, two neural network types coexist: the target neural network (QN-target), and the evaluation neural network (QN-eval). The DQN unfolds through

two distinct phases: an observation period and a training period. While the fundamental steps remain consistent between the two stages, the sole contrast lies in the neural network training. Throughout the observation period, the Q-net neural network remains unaltered, solely capturing the replay memory of the scheduling sequence, which includes the reward value. Conversely, in the training phase, the neural network undergoes periodic updates. The target neural network, QN-target, is updated to estimate Q-target at intervals defined by the T-renew interval, while the evaluation neural network, QN-eval, is trained to estimate Q-eval based on the T-gap interval. Typically, T-renew is a multiple integer of T-gap, as observed in the simulation where it is set at five times the T-gap interval. Figure 2 gives the key step of the DQN method.

Simulations and comparisons

Section 5.1 gives the parameters and the simulation environment used in the simulation, Sect. 5.2 gives the simulation result, and a deep discussion has been given to it.

We will compare our method with OBP (offloading based on priorities) [44], BBO (biogeography-based optimization) [45], GWO (grey wolf optimizer) [46], and DPP (Drift-Plus-Penalty) [47]. OBP gave priorities to tasks according to the energy consumption of different locations and task urgency. [46], OBP gave priorities to tasks according to the energy consumption of different locations and task urgency. It gave a reference working state to ensure that each task has the minimum energy

consumption. BBO [45] is a new multi-objective strategy based on biogeography-based optimization to satisfy the execution time, energy consumption, and cost. GWO [46] employs a grey wolf optimizer to optimize the offloading result of the mobile edge devices. DPP [47] takes the offloading problems as a mixed integer programming problem, and design a heuristic to solve the problem. To stratify our environment, we assume that the three methods always meet the total available energy of the medical device. Our method is proposed for the medical mobile device, called it “O-Med”.

Simulation environment

All the parameters are given in Tables 2, 3 and 4. Table 2 is the related metrics about the simulation environment. Table 3 and 4 are the DVFS model and other metrics of the medical mobile device and cloud resource. The initial available energy of the medical mobile device it a rand between $[minen, maxen]$, where $minen$ and $maxen$ are the minimum and the maximum required energy of all tasks.

In each simulation, there are 20,000 tasks totally. The number of instructions is a random number in $[100, 200000]$ *MI* (Million instructions). The file size of input files (same as the output files) is a random number in $[10, 1000]$ *M*. The deadline of a task is a random in $[1, 5]$ minimum execution time. The average arrival rate is changed from 50 to 100 with a step of 5.

Our simulation tool is matlab (2023), the OS is windows 10. The simulation environment is Intel(R) Core(TM) i7-4790 CPU, 16G RAM.

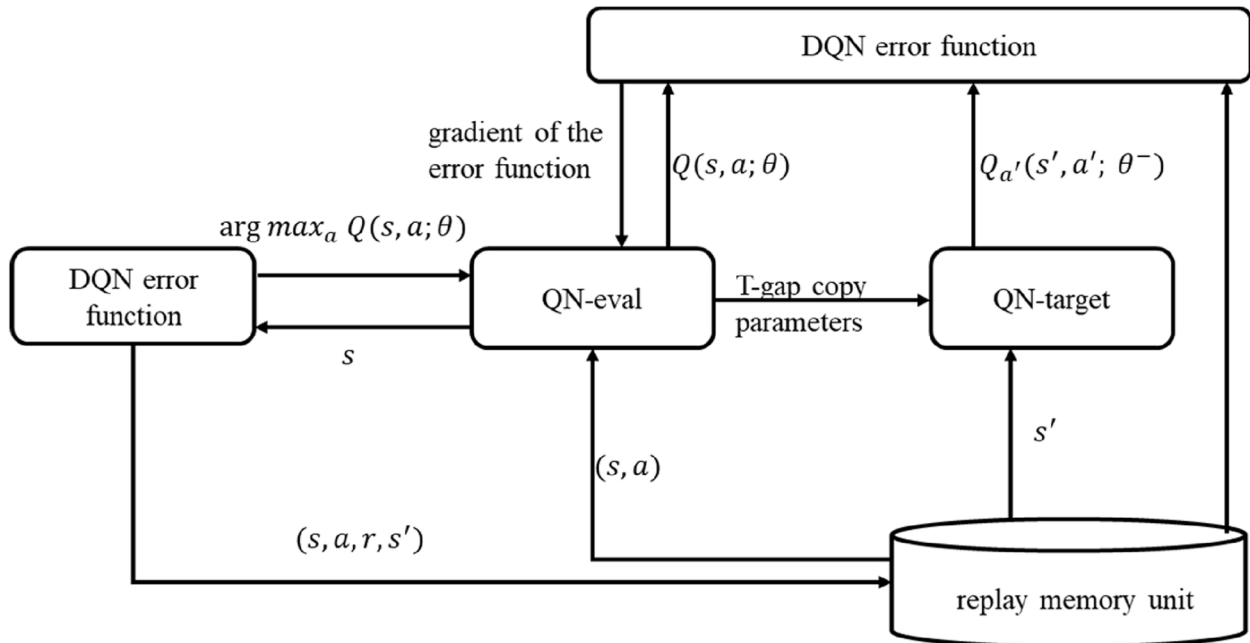


Fig. 2 DQN for offloading tasks

Table 2 Simulation parameters

Parameters	Values
Computing speed of the mobile device	1000 (4 cores) MHz
Computing speed of cloud	2500 (8 cores) MHz
Power consumption of the working mobile device	2W
Power consumption of the idle mobile device	0.001W
Power consumption of receiving data	0.05W
Power consumption of sending data	0.1W
Receiving data rate (from the cloud to the mobile device)	5~20 M/s
Seeding data rate (from the mobile device to the cloud)	5~10 M/s

Table 3 DVFS model for mobile device

Computing frequency	Working voltage	Power consumption
0.84 GHz	1.05 V	0.64W
0.88 GHz	1.10 V	0.74W
0.92 GHz	1.15 V	0.84W
0.96 GHz	1.15 V	0.88W
1.0 GHz	1.2 V	1 W

Table 4 DVFS model for remote Cloud resources

Computing frequency	Working voltage	Power consumption
1.8 GHz	1.05 V	1.10W
2.0 GHz	1.10 V	1.37W
2.1 GHz	1.15 V	1.54W
2.2 GHz	1.15 V	1.6W
2.5 GHz	1.2 V	2.0 W

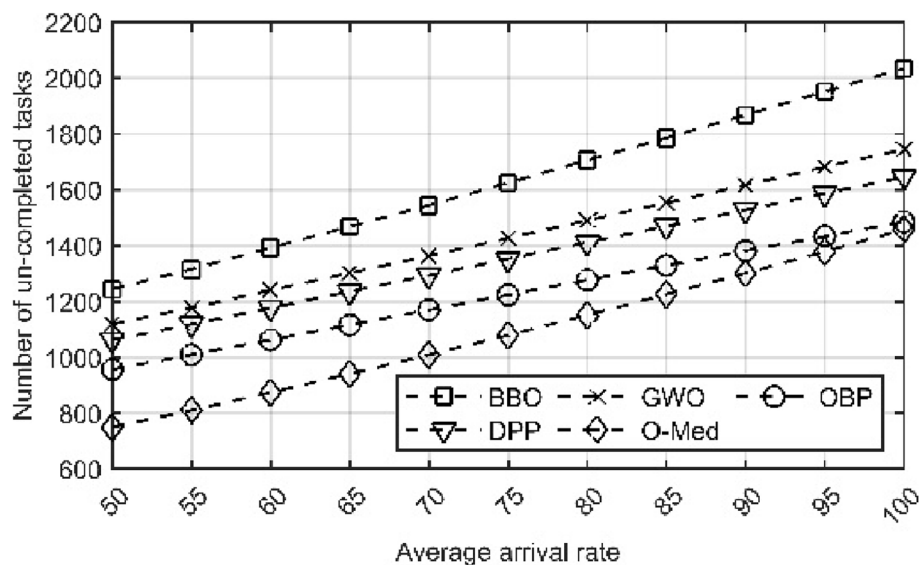
Result and discussion

In this section, we will compare the four methods according to three aspects: (1) NUT: number of un-completed tasks, (2) AEC: Average energy consumption, and (3) AET: Average execution time.

Number of un-completed tasks

Figure 3 is the NUT of BBO, GWO, OBP, and O-Med when the arrival rate is changed from 50 to 100 with a step of 5. In general, all methods have an increasing trend in NUT with the increase of the AAR (average arrival rate). O-med always has the lowest of NUT, followed by OBP, GWO, and BBO. The average of NUT of BBO, GWO, OBP, DPP and O-Med, is $1.6299e+03$, $1.4282e+03$, $1.2219e+03$, $1.352 e+03$, $1.0890e+03$, respectively. Compared to NUT of O-Med, BBO, GWO, OBP, and DPP improves 49.67%, 31.15%, 12.20%, and 24.20%, respectively. O-Med not only consider the deadline of tasks, it also try to make the best tradeoff between the short-term and long-term benefit, so is has the minimum value in NUT.

Figures 3 and Fig. 4 shows that O-Med has finished more tasks than other methods. A question is whether O-Med drops some small tasks (tasks with a small number in the instructions and file size, or both of two aspects) in the simulation. Figure 5 is the total file size of the input files and output files and Fig. 6 is the total number of instructions of the completed tasks. Figure 5 and Fig. 6 show that O-Med always has the largest value in the TFS (Total file size of all completed files) and TI (Total number of instructions of all completed files). Compared to TFS of BBO, GWO, OBP, and DPP, O-Med has an improvement of 2.06%, 4.09%, 0.61%, and

**Fig. 3** Number of un-completed tasks

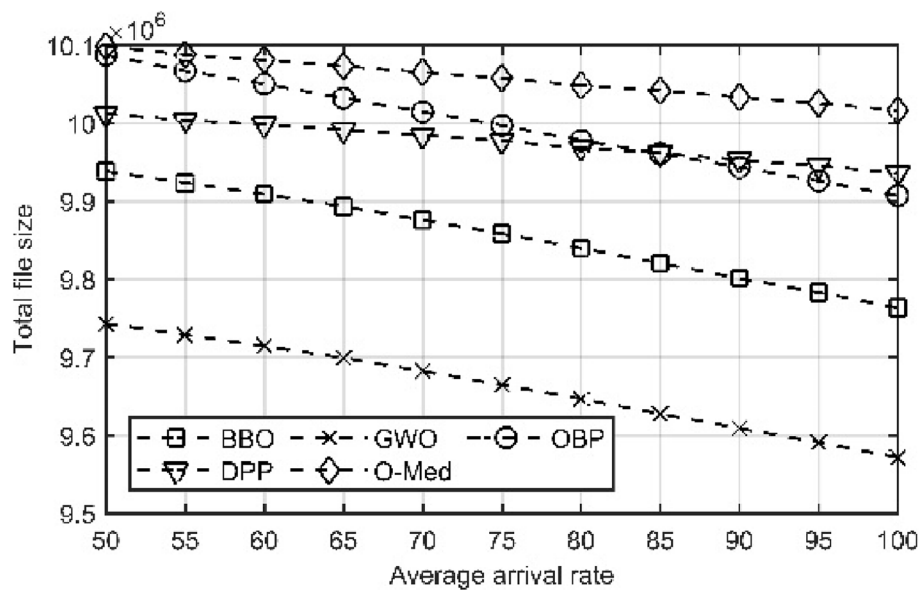


Fig. 4 Total file size of completed tasks

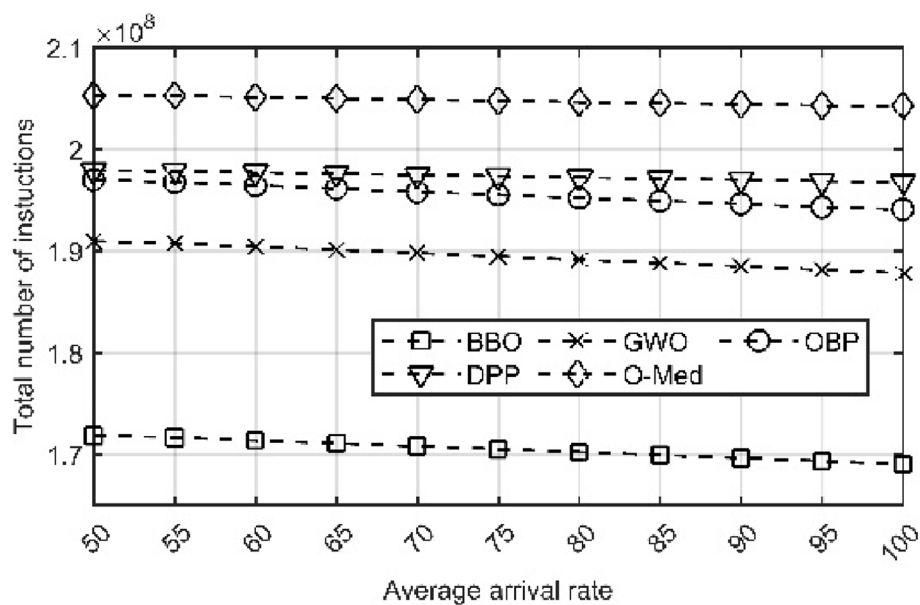


Fig. 5 Total number of instructions

0.82%, respectively. Compared to TI of BBO, GWO, OBP, and DPP, O-Med has an improvement of $2.23(e+6)$, $4.35(e+6)$, and $6.70(e+5)$, about 2.11%, 8.10%, 4.73%, and 4.39, respectively. This proves that O-Med has an advantage in NUT, not because it drops some small tasks. O-Med performs best in the UNT because DQN is adaptive to the energy supply to meet most of the tasks. BBO and GWO may not involve in a global optimization. Though BBO gives tasks different priorities, sometimes,

an execution of a task may make more tasks cannot be completed before the deadline.

Average energy consumption

Figure 6 is the AEC (average energy consumption) of the four methods when they have various AARs. Generally, the AEC of all methods increases gradually with the improvement of AAR. The average AEC of the BBO, GWO, OBP, DPP, and O-Med is 530.50, 548.26, 482.67,

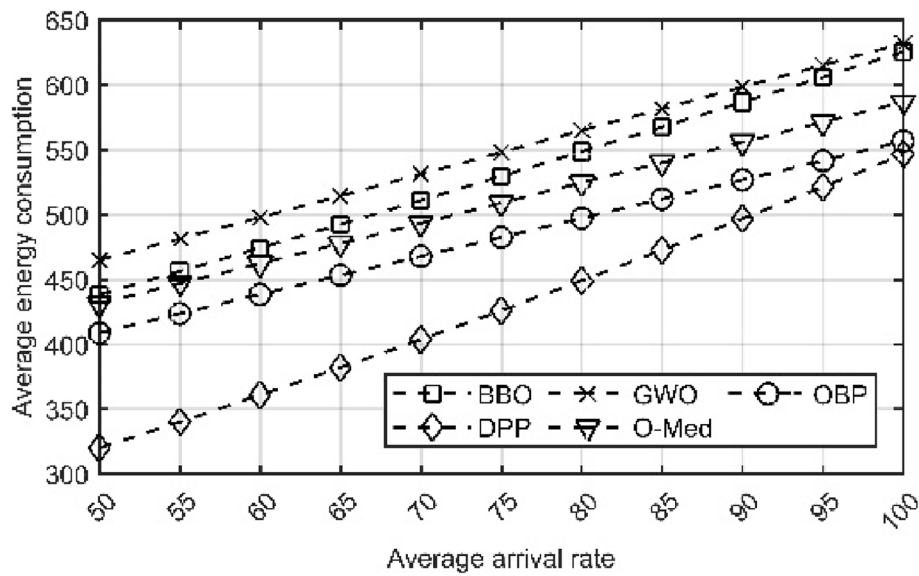


Fig. 6 Average energy consumption of completed tasks

508.90, and 428.65, respectively. O-Med has the lowest value in AEC at any arrival rates, followed by OBP, BBO, and GWO. Compared to AEC OBP, BBO, GWO, and DPP, O-Med decrease 101.85, 119.61, 54.02 and 80.25, about 23.76%, 27.90%, 12.60% and 18.72%. BBO and GWO only have a little difference in AEC. OBP gives priorities to different tasks. One problem involved in OBP is the tasks affect each other. The priorities do not always reflect the order of the execution of tasks. If a large task consumes a lot of resources, the leaving tasks may have a high priorities to ensure they can be completed. So, OBP has problems under some cases. BBO and GWO always try to find the best solution. Under some cases, they may both trapped in a local optimization. DPP is a heuristics, and some times, it just considers the scheduling for now, does not consider the scheduling influences the future tasks, so, it does not have minimum in AEC.

Average execution time

Figure 7 is the AET (Average execution time) of BBO, GWO, OBP and O-Med under various AARs. All methods have an increasing trend when the AAR is increased. This is all methods that spend more time awaiting the execution, such increasing the execution time. The increasing order of AET of the four methods is: O-Med, OBP, GWO, and BBO. O-Med always has the lowest value in the four methods. Compared to OBP, GWO, and BBO, AET of O-Med decreases 116.30, 66.70, 27.70, and 15.88, about 6.32%, 3.62%, 1.50%, and 0.86%, respectively. O-Med considers the multiple targets as the score of DQN, so it also has good performance AET.

In summary, our proposed method O-Med has good performance in various aspects. O-Med performs well because the score of DQN take all the scheduling targets. At the same time, O-Med maximize the score for long-term. The Grey Wolf Optimizer (GWO) is a nature-inspired optimization algorithm that is based on the social hierarchy and hunting behavior of grey wolves. While it is used for the offloading problem, it has limited adaptability and extensibility, same to GWO. BBO also fails to solve the constraint for the offloading problem, so it has the highest value in NUT.

Conclusions

In this paper, we focus on the offloading problem in the medical mobile device while meeting the available energy of the medical mobile device. We consider the energy consumption at various places (on the mobile device or the remote cloud) and the different working states (DVFS for the medical mobile device and the cloud resource). First of all, based on the DVFS model, we rank those working states and designed the reward based on the ranking. Then, we use a DQN method to offload tasks for the medical mobile devices. Simulations show that our proposed method has an advantage in the number of un-completed tasks, at the same time, it has the lowest value in the execution time and the energy consumption. Different to the past work, we also consider the energy efficiency problem of the MD, so, our proposed method can save energy to others. This not just for the medical mobile device, for all mobile device, it also can save energy. For MMD, we

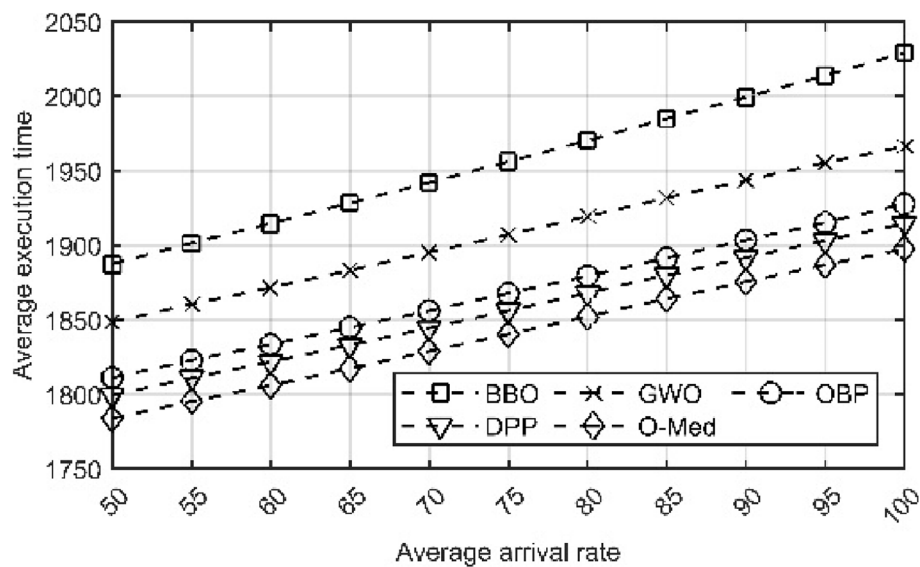


Fig. 7 Average execution time

also consider the particularity of the work of doctors in a hospital, so the proposed method may meet the requirement in the hospital. As a future work, in some hospitals, there are many doctors and nurses, and the system may have a higher system load, compared to this environment, we need to take into account cloud resources and the medical mobile device at the same time.

Authors' contributions

Conceptualization: Junwen Lu, Min Zhao Methodology: Junwen Lu Software: Min Zhao Validation: Junwen Lu, Min Zhao Formal Analysis: Junwen Lu Investigation: Min Zhao Resources: Min Zhao Data Curation: Junwen Lu Writing – Original Draft: Junwen Lu Writing – Review & Editing: Junwen Lu, Min Zhao Visualization: Min Zhao Funding Acquisition: Junwen Lu Both authors read and approved the final manuscript.

Funding

No funding was used in this study.

Availability of data and materials

No underlying data was collected or produced in this study.

Data Availability

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 17 May 2024 Accepted: 5 August 2024

Published online: 12 August 2024

References

- Kar B, Lin YD, Lai YC (2023) Cost optimization of omnidirectional offloading in two-tier cloud-edge federated systems. *J Netw Comput Appl*. 215:103630. <https://doi.org/10.1016/j.jnca.2023.103630>
- Hao Y, Wang Q, Ma T, Du J, Cao J (2023) Energy allocation and task scheduling in edge devices based on forecast solar energy with meteorological information. *J Parallel Distrib Comput* 177:171–181. <https://doi.org/10.1016/j.jpdc.2023.03.005>
- Kumar D, Baranwal G, Shankar Y, Vidyarthi DP (2022) A survey on nature-inspired techniques for computation offloading and service placement in emerging edge technologies. 25:5. Springer US. <https://doi.org/10.1007/s11280-022-01053-y>
- Han S (2020) Congestion-aware WiFi offload algorithm for 5G heterogeneous wireless networks. *Comput Commun* 164(September):69–76. <https://doi.org/10.1016/j.comcom.2020.10.006>
- Wang E, Wang H, Dong PM, Xu YB, Yang YJ (2022) Distributed game-theoretical D2D-enabled task offloading in mobile edge computing. *J Comput Sci Technol* 37(4):919–941. <https://doi.org/10.1007/s11390-022-2063-3>
- Mustafa E et al (2022) Joint wireless power transfer and task offloading in mobile edge computing: a survey. *Cluster Comput* 25(4):2429–2448. <https://doi.org/10.1007/s10586-021-03376-3>
- Pang S et al (2023) A Stackelberg game scheme for pricing and task offloading based on idle node-assisted edge computational model. *Simul Model Pract Theory*. 124(December 2022):102725. <https://doi.org/10.1016/j.simpat.2023.102725>
- Yan P, Choudhury S (2021) Deep Q-learning enabled joint optimization of mobile edge computing multi-level task offloading. *Comput Commun* 180(June):271–283. <https://doi.org/10.1016/j.comcom.2021.09.028>
- Hao Y, Wang Q, Cao J, Ma T, Du J, Zhang X (2022) Interval grey number of energy consumption helps task offloading in the mobile environment. *ICT Express*. <https://doi.org/10.1016/j.icte.2022.03.005>
- Hao Y, Cao J, Wang Q, Du J (2021) Energy-aware scheduling in edge computing with a clustering method. *Futur Gener Comput Syst* 117:259–272. <https://doi.org/10.1016/j.future.2020.11.029>
- Zhang Q, Lin X, Hao Y, Cao J (2020) Energy-aware scheduling in edge computing based on energy internet. *IEEE Access* 8:229052–229065. <https://doi.org/10.1109/ACCESS.2020.3044932>
- Chen W, Chen Y, Liu J (2023) Service migration for mobile edge computing based on partially observable Markov decision processes. *Comput Electr Eng*. 106(December 2022):108552. <https://doi.org/10.1016/j.compeleceng.2022.108552>

13. Lin J, Huang S, Zhang H, Yang X, Zhao P (2023) A deep-reinforcement-learning-based computation offloading with mobile vehicles in vehicular edge computing. *IEEE Internet Things J* 10(17):15501–15514. <https://doi.org/10.1109/JIOT.2023.3264281>
14. Qiu J, Wang R, Chakrabarti A, Guerin R, Lu C (2022) Adaptive edge offloading for image classification under rate limit. *IEEE Trans Comput Des Integr Circuits Syst* 41(11):3886–3897. <https://doi.org/10.1109/TCAD.2022.3197533>
15. Tang H, Wu H, Qu G, Li R (2023) Double deep Q-network based dynamic framing offloading in vehicular edge computing. *IEEE Trans Netw Sci Eng* 10(3):1297–1310. <https://doi.org/10.1109/TNSE.2022.3172794>
16. Wu G, Chen X, Gao Z, Zhang H, Chen X (2023) Journal Pre-proof. <https://doi.org/10.1016/j.jpdc.2023.104775>
17. Mwasinga LJ, Le D-T, Raza SM, Challa R, Kim M, Choo H (2023) RASM: Resource-Aware Service Migration in edge computing based on deep reinforcement learning. *J Parallel Distrib Comput* 182:104745. <https://doi.org/10.1016/j.jpdc.2023.104745>
18. Li C, Cai Q, Luo Y (2021) Multi-edge collaborative offloading and energy threshold-based task migration in mobile edge computing environment. *Wirel Netw* 27(7):4903–4928. <https://doi.org/10.1007/s11276-021-02776-y>
19. Liu S, Yang Q, Zhang S, Wang T, Xiong NN (2022) MIDP: An MDP-based intelligent big data processing scheme for vehicular edge computing. *J Parallel Distrib Comput* 167:1–17. <https://doi.org/10.1016/j.jpdc.2022.04.013>
20. Ben Dhaou I, Ebrahimi M, Ben Ammar M, Bouattour G, Kanoun O (2021) Edge devices for internet of medical things: Technologies, techniques, and implementation. *Electron. J* 10(17):1–20. <https://doi.org/10.3390/electronics10172104>
21. Zhao Z, Liu X, Hao K, Zheng T, Xu J, Cui S (2022) PIS-YOLO: real-time detection for medical mask specification in an edge device. *Comput Intell Neurosci* 2022:1–11. <https://doi.org/10.1155/2022/6170245>
22. Lebre R, Silva LB, Costa C (2020) A cloud-ready architecture for shared medical imaging repository. *J Digit Imaging* 33(6):1487–1498. <https://doi.org/10.1007/s10278-020-00373-7>
23. Xu F, Xie Y, Sun Y, Qin Z, Li G, Zhang Z (2022) Two-stage computing offloading algorithm in cloud-edge collaborative scenarios based on game theory. *Comput Electr Eng* 97(June 2021):107624. <https://doi.org/10.1016/j.compeleceng.2021.107624>
24. Fresa A, Champati JP (2023) Offloading algorithms for maximizing inference accuracy on edge device in an edge intelligence system. *IEEE Trans Parallel Distrib Syst* 34(7):2025–2039. <https://doi.org/10.1109/TPDS.2023.3267458>
25. Wang M, Zhang Y, He X, Yu S (2023) Joint scheduling and offloading of computational tasks with time dependency under edge computing networks. *Simul Model Pract Theory* 129(September):102824. <https://doi.org/10.1016/j.simpat.2023.102824>
26. Pacheco RG, Couto RS, Simeone O (2022) On the impact of deep neural network calibration on adaptive edge offloading for image classification. *J Netw Comput Appl* 217(September):2023. <https://doi.org/10.1016/j.jnca.2023.103679>
27. Zhang Z, Zeng F (2023) Efficient task allocation for computation offloading in vehicular edge computing. *IEEE Internet Things J* 10(6):5595–5606. <https://doi.org/10.1109/JIOT.2022.3222408>
28. Hao Y, Cao J, Ma T, Ji S (2019) Adaptive energy-aware scheduling method in a meteorological cloud. *Futur Gener Comput Syst* 101:1142–1157. <https://doi.org/10.1016/j.future.2019.07.061>
29. Singh R, Singh S (2023) Edge AI: a survey. *Internet Things Cyber-Physical Syst* 3(February):71–92. <https://doi.org/10.1016/j.iotcps.2023.02.004>
30. Hortelano D et al (2023) A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems. *J Netw Comput Appl* 216(October 2022):103669. <https://doi.org/10.1016/j.jnca.2023.103669>
31. Huang H, Ye Q, Zhou Y (2022) Deadline-aware task offloading with partially-observable deep reinforcement learning for multi-access edge computing. *IEEE Trans Netw Sci Eng* 9(6):3870–3885. <https://doi.org/10.1109/TNSE.2021.3115054>
32. Peng X et al (2023) Deep reinforcement learning for shared offloading strategy in vehicle edge computing. *IEEE Syst J* 17(2):2089–2100. <https://doi.org/10.1109/JSYST.2022.3190926>
33. Ju Y et al (2023) Joint secure offloading and resource allocation for vehicular edge computing network: a multi-agent deep reinforcement learning approach. *IEEE Trans Intell Transp Syst* 24(5):5555–5569. <https://doi.org/10.1109/TITS.2023.3242997>
34. Zhu X, Zhang T, Zhang J, Zhao B, Zhang S, Wu C (2023) Deep reinforcement learning-based edge computing offloading algorithm for software-defined IoT. *Comput Networks* 235(September). <https://doi.org/10.1016/j.comnet.2023.110006>
35. Ge P, Zhao J, Zhang H, Zou D, Wang M (2023) Green hybrid energy harvesting for intelligent mobile edge computing in internet of things. *Phys Commun* 61:102171. <https://doi.org/10.1016/j.phycom.2023.102171>
36. Heidari A, Navimipour NJ, Jamali MAJ, Akbarpour S (2023) A green, secure, and deep intelligent method for dynamic IoT-edge-cloud offloading scenarios. *Sustain Comput Informatics Syst* 38(December 2022):100859. <https://doi.org/10.1016/j.suscom.2023.100859>
37. Huang L, Feng X, Zhang C, Qian L, Wu Y (2019) Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing. *Digit Commun Networks* 5(1):10–17. <https://doi.org/10.1016/j.dcan.2018.10.003>
38. Ge H et al (2022) Task Offloading Algorithm in Edge Computing Based on DQN. *Proc-2022 4th Int Conf Nat Lang Process ICNLP*. 2022:482–488. <https://doi.org/10.1109/ICNLP55136.2022.00088>
39. Feng T, Wang B, Zhao HT, Zhang T, Tang J, Wang Z (2021) Task Distribution Offloading Algorithm Based on DQN for Sustainable Vehicle Edge Network. *Proc 2021 IEEE Conf Netw Softwarization Accel Netw Softwarization Cogn Age NetSoft* 2021:430–436. <https://doi.org/10.1109/NetSoft51509.2021.9492640>
40. Klonoff DC (2017) Fog computing and edge computing architectures for processing data from diabetes devices connected to the medical internet of things. *J Diabetes Sci Technol* 11(4):647–652. <https://doi.org/10.1177/1932296817717007>
41. Yang CT, Shih WC, Chen LT, Kuo CT, Jiang FC, Leu FY (2015) Accessing medical image file with co-allocation HDFS in cloud. *Futur Gener Comput Syst* 43–44:61–73. <https://doi.org/10.1016/j.future.2014.08.008>
42. Viana-Ferreira C, Guerra A, Silva JF, Matos S, Costa C (2017) An intelligent cloud storage gateway for medical imaging. *J Med Syst* 41(9). <https://doi.org/10.1007/s10916-017-0790-8>
43. Asghari P, Rahmani AM, Haj Seyyed Javadi H (2019) A medical monitoring scheme and health-medical service composition model in cloud-based IoT platform. *Trans Emerg Telecommun Technol* 30(6). <https://doi.org/10.1002/ett.3637>
44. Hao Y, Cao J, Wang Q, Ma T (2021) Energy-aware offloading based on priority in mobile cloud computing. *Sustain Comput Informatics Syst* 31(March):100563. <https://doi.org/10.1016/j.suscom.2021.100563>
45. Li H, Zheng P, Wang T, Wang J, Liu T (2023) A multi-objective task offloading based on BBO algorithm under deadline constrain in mobile edge computing. *Cluster Comput* 26(6):4051–4067. <https://doi.org/10.1007/s10586-022-03809-7>
46. Wang X, Zhou Z, Chen H, Zhang Y (2022) Task offloading and power assignment optimization for energy-constrained mobile edge computing. *Proc. - 2021 9th Int Conf Adv Cloud Big Data CBD* 2021. 302–307. <https://doi.org/10.1109/CBD54617.2021.00058>
47. Liu Y, Mao Y, Shang X, Liu Z, Yang Y (2023) Energy-aware online task offloading and resource allocation for mobile edge computing. In: *Proceedings - International Conference on Distributed Computing Systems*, IEEE 339–349. <https://doi.org/10.1109/ICDCS57875.2023.00073>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.